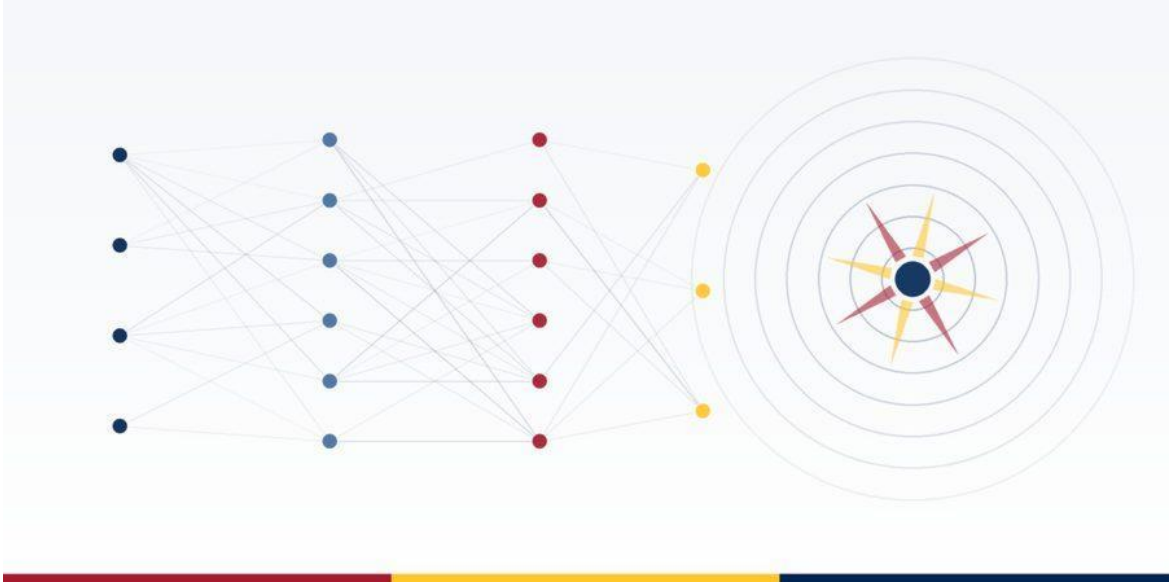
QUEEN'S UNIVERSITY

ELEC 475

Computer Vision with Deep Learning

Lab 2 - Deepening the Network: Full AlexNet, Normalization Study, & Data Augmentation



Submitted by

Parisa Rahmani

Student ID: 20627491

Tien Nguyen

Student ID: 20629617

2026-10-17

Instructor: Dr. Michael Greenspan

Architectural Specifications

All models take a 3 * 64 * 64 RGB input and output K class (K = 10 for CIFAR-10 and K = 200 for Tiny ImageNet-200). CIFAR-10 images are resized from 32 * 32 to 64 * 64; to be the same size as Tiny ImageNet images that are 64 * 64. The final layer has no activation, since the cross-entropy loss applies SoftMax internally.

The spatial output size of each convolution and pooling layer is $N_o = \lfloor \frac{N_i - k + 2P}{S} \rfloor + 1$. Parameter counts follow $C_{in} * k^2 * C_out + C_out$ for convolutions, $n_{in} * n_out + n_out$ for fully connected (FC) layers, and $2C$ (γ, β) for BatchNorm. The receptive field (RF) is computed recursively as $RF_l = RF_{l-1} + (k_l - 1) * J_(l - 1)$, with jump $J_l = J_{l-1} * s_l$ and $RF_0 = J_0 = 1$.

Base LeNet-5 and Modernized LeNet

The base LeNet uses two 5 * 5 convolution stages with Tanh activations and non-overlapping 2 * 2 average pooling first. Then it uses three fully connected layers.
The modernized LeNet makes three changes:

- Tanh is replaced with ReLU.
- 2 * 2 average pooling is replaced with overlapping 3 * 3, stride-2 max pooling.
- Padding P = 2 is added to both convolutions.

Table 1. Layer-by-layer specification

Layer	LeNet-5 output	Params	Modern LeNet output	Params
Input	3*64*64	–	3*64*64	–
Conv1 5*5, 6 filters	6*60*60 (P=0)	456	6*64*64 (P=2)	456
Activation	Tanh	–	ReLU	–
Pool1	Avg 2*2/2 -> 6*30*30	–	Max 3*3/2 -> 6*31*31	–
Conv2 5*5, 16 filters	16*26*26	2,416	16*31*31	2,416
Pool2	16*13*13	–	16*15*15	–
Flatten	2,704	–	3,600	–
FC1 -> 120	120	324,600	120	432,120
FC2 -> 84	84	10,164	84	10,164
FC3 -> K	10	850	10	850
Total parameters		338,486		446,006
MACs / FLOPs		3.59 M / 7.17 M		4.59M / 9.19M
Receptive field		16 * 16 px		19 * 19 px

Parameter breakdowns: In both models, FC1 holds 96-97% of all parameters, while the convolutions hold under 1%, because each filter's weights are shared across every spatial position. The only parameter difference between the two models is FC1: overlapping pooling with padding leaves a 15 * 15 map instead of 13 * 13, so the flattened vector grows from 16*13*13 = 2704 to 16*15*15= 3600 (+107,520 weights). The final receptive field covers only 16 * 16 px (LeNet-5) or 19 * 19 px (Modern LeNet) of the 64 * 64 input, so each final feature sees a local patch rather than the whole object.

*Table 2. Receptive-Field Calculations. LeNet-5 (input 64 * 64)*

Layer	k	xs	RF = RF_old + (k - 1) * J_old	J = J_old * s
Input	–	–	1	1
conv1	5	1	1 + (5-1) * 1 = 5	1*1 = 1
avgpool1	2	2	5 + (2-1) * 1 = 6	1*2 = 2
conv2	5	1	6 + (5-1) * 2 = 14	2*1 = 2
avgpool2	2	2	14 + (2-1) * 2 = **16**	2*2 = 4

Final RF = 16 * 16

*Table 3. Receptive-Field Calculations. Modern LeNet-5 (input 64 * 64)*

Layer	k	s	RF = RF_old + (k - 1) * J_old	J = J_old * s
Input	–	–	1	1
conv1	5	1	1 + (5-1) * 1 = 5	1*1 = 1
Maxpool1	3	2	5 + (3-1) * 1 = 7	1*2 = 2
conv2	5	1	7 + (5-1) * 2 = 15	2*1 = 2
Maxpool2	3	2	15 + (3-1) * 2 = **19**	2*2 = 4

Final RF = 19 * 19 - the larger (overlapping) 3 * 3 pools add 3 px over LeNet-5.

Full AlexNet-64

AlexNet-64 adapts AlexNet to 64 * 64 inputs. It has five convolution stages with widening channels (64 -> 192 -> 384 -> 384 -> 256), overlapping 3 * 3 / 2 max pooling after stages 1, 2 and 5, and a classifier with Dropout (p = 0.5) before FC1 and FC2. A stride-2 first convolution halves the resolution immediately, which reduces the compute of every later layer by about 4 times.

Table 4. AlexNet-64 specification (K = 10).

Stage	Layer	Output	Params	BN params
1	Conv 5 * 5/2, P=2, 64 -> Norm -> ReLU	64 * 32 * 32	4,864	128
	MaxPool 3 * 3/2	64 * 15 * 15	–	–
2	Conv 5 * 5/1, P=2, 192 -> Norm -> ReLU	192 * 15 * 15	307,392	384
	MaxPool 3 * 3/2	192 * 7 * 7	–	–
3	Conv 3 * 3/1, P=1, 384 -> (BN) -> ReLU	384 * 7 * 7	663,936	768
4	Conv 3 * 3/1, P=1, 384 -> (BN) -> ReLU	384 * 7 * 7	1,327,488	768
5	Conv 3 * 3/1, P=1, 256 -> (BN) -> ReLU	256 * 7 * 7	884,992	512
	MaxPool 3 * 3/2 -> Flatten	256 * 3 * 3 -> 2,304	–	–
Head	Dropout 0.5 -> FC 2304 -> 1024 -> ReLU	1,024	2,360,320	–
	Dropout 0.5 -> FC 1024 -> 1024 -> ReLU	1,024	1,049,600	–
	FC 1024 -> K	10	10,250	–
Total	none / LRN: 6,608,842		BN: 6,611,402	+2,560

Parameter breakdown. Convolutions hold 3,188,672 parameters (48%) and FC layers 3,420,170 (52%). Unlike LeNet, where FC1 dominates, capacity has moved into the feature extractor. MACs / FLOPs are 218.35 M / 436.70 M (none, LRN) and 218.98 M / 437.97 M (BN). The handout's nominal ≈ 6.3 M parameters and 250–257 MFLOPs differ from what its own layer specification yields (6.61 M, 437 MFLOPs), which our implementation reproduces exactly.

AlexNet-64 specification (K = 10). "Norm" is LocalResponseNorm (size = 5, $\alpha = 10^{-4}$, $\beta = 0.75$, k = 2) in the lrn variant, BatchNorm2d in the bn variant, and Identity in the none variant. As in the original 2012 paper, LRN is applied after stages 1–2 only; "(BN)" marks stages 3–5, which are normalized only in the bn variant. BN parameters are present only in the bn variant.

*Table 5. Receptive-Field Calculations. AlexNet-64 (input 64 * 64)*

Layer	k	s	RF = RF_old + (k - 1)*J_old	J = J_old*s
Input	–	–	1	1
conv1	5	2	1 + (5-1) * 1 = 5	1 * 2 = 2
pool1	3	2	5 + (3-1) * 2 = 9	2 * 2 = 4
conv2	5	1	9 + (5-1) * 4 = 25	4 * 1 = 4
pool2	3	2	25 + (3-1) * 4 = 33	4 * 2 = 8
conv3	3	1	33 + (3-1) * 8 = 49	8 * 1 = 8
conv4	3	1	49 + (3-1) * 8 = 65	8 * 1 = 8
conv5	3	1	65 + (3-1) * 8 = 81	8 * 1 = 8
pool3 (final)	3	2	**81 + (3-1) * 8 = 97**	**8 * 2 = 16**

Final RF = 97 * 97 > 64 * 64 input - every final unit sees the whole image (the excess falls on zero padding).

PyTorch Model Zoo AlexNet baseline

Table 6. Zoo AlexNet - designed for 224×224; shapes for our 64×64 input

Layer	Output	Params
Conv 11 * 11/4, P=2, 64 -> ReLU -> MaxPool 3 * 3/2	64 * 15 * 15 -> 64 * 7 * 7	23,296
Conv 5 * 5, P=2, 192 -> ReLU -> MaxPool 3 * 3/2	192 * 7 * 7 -> 192 * 3 * 3	307,392
Conv 3 * 3: 384 / 256 / 256 -> ReLU	256 * 3 * 3	663,936 + 884,992 + 590,080
MaxPool 3 * 3/2	256 * 1 * 1	–
AdaptiveAvgPool2D (6 * 6) -> Flatten	9,216	–
Dropout -> FC 9216 -> 4096 -> ReLU	4,096	37,752,832
Dropout -> FC 4096 -> 4096 -> ReLU	4,096	16,781,312
FC 4096 -> K	10	40,970
Total (MACs / FLOPs: 94.10 M / 188.20 M)		57,044,810

Parameter breakdown. The three FC layers hold 54.6 M of the 57.0 M parameters (96%). Because the architecture targets 224 * 224 images, its stride-4 first convolution and three pooling stages reduce a 64 * 64 input to a single 1*1 position. `AdaptiveAvgPool2d((6, 6))` then replicates it to 6 * 6, so FC1's 9,216 inputs are 36 copies of the same 256 values. The model has 8.6 * more parameters than AlexNet-64 but only 0.43 * its FLOPs.

Tiny ImageNet (K = 200). Only the final layer depends on K. The parameter counts become: LeNet-5 354,636; Modern LeNet 462,156; AlexNet-64 6,803,592 (none / LRN) and 6,806,152 (BN); Zoo AlexNet 57,823,240.

Table 7. Receptive-Field Calculations. Zoo AlexNet

Layer	k	s	RF = RF_old + (k - 1)*J_old	J = J_old*s
Input	–	–	1	1
conv1	11	4	1 + (11-1) * 1 = 11	1 * 4 = 4
pool1	3	2	11 + (3-1) * 4 = 19	4 * 2 = 8
conv2	5	1	19 + (5-1) * 8 = 51	8 * 1 = 8
pool2	3	2	51 + (3-1) * 8 = 67	8 * 2 = 16
conv3	3	1	67 + (3-1) * 16 = 99	16 * 1 = 16
conv4	3	1	99 + (3-1) * 16 = 131	16 * 1 = 16
conv5	3	1	131 + (3-1) * 16 = 163	16 * 1 = 16
pool3 (final)	3	2	**163 + (3-1) * 16 = 195**	**16 * 2 = 32**

Final RF = 195 * 195 > 64 * 64 input - every final unit sees the whole image (the excess falls on zero padding).

Empirical Results and Figures

All runs used train.py on a Colab NVIDIA Tesla T4 with Adam (learning rate 10^{-3}), batch size 128, cross-entropy loss and global seed 42. Each run lasted 10 epochs, except the 15-epoch augmentation study. Accuracy and macro F1 are reported at the best validation epoch. Tiny ImageNet was evaluated on its official 10,000-image validation set. The starter loader validated on the training folder, which we corrected.

Master Ablation Study Summary Tables

Table 8. Architectural configuration and computational complexity. Latency is at batch size 1, the mean of two compare_models.py runs

Model	Conv depth	Normalization & regularization	Data aug.	Params (M)	MFLOPs	Mean epoch (s)	Latency (ms)
Base LeNet-5	2	None	None	0.338	7.17	12.97	0.35
Modern LeNet	2	None. ReLU + overlap. MaxPool	None	0.446	9.19	12.87	0.38
AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.70	15.53	2.00
AlexNet + LRN	5	LRN + Dropout	None	6.609	436.70	22.59	1.91
AlexNet + BN	5	BatchNorm2d + Dropout	None	6.611	437.97	19.90	1.46
AlexNet + BN + Aug	5	BatchNorm2d + Dropout	Crop / Flip / Jitter	6.611	437.97	30.71	1.46
PyTorch Zoo AlexNet	5	Dropout (AdaptiveAvgPool2d)	None	57.045	188.20	19.76	1.92

Table 9. Predictive performance (best validation epoch, 10 epochs)

Model	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1(%)
Base LeNet-5	55.23	55.29	12.41	10.80
Modern LeNet	66.65	66.82	20.77	18.91
AlexNet (No Norm)	73.52	73.48	16.46	13.68
AlexNet + LRN	75.14	75.26	0.50 *	0.00 *
AlexNet + BN	79.93	80.06	30.15	28.25
AlexNet + BN + Aug	78.73	79.01	29.20	27.61
PyTorch Zoo AlexNet	65.85	65.61	11.02	8.21

* Training collapsed: loss stayed at ln(200) = 5.298 for all 10 epochs.

Extended Ablation Exploration

Table 10. Extended ablation exploration (Δ = variant - reference, best validation accuracy in percentage points; gap = final training - final validation accuracy).

Change (reference -> variant)	Dataset / epochs	Reference	Variant	Δ accuracy	Other effect
Tanh + AvgPool -> ReLU + overlapping MaxPool (LeNet)	CIFAR-10 / 10	55.23	66.65	+11.42	same epoch time
	Tiny / 10	12.41	20.77	+8.36	same epoch time
No norm -> LRN (AlexNet-64)	CIFAR-10 / 10	73.52	75.14	+1.62	+46% epoch time
	Tiny / 10	16.46	0.50	-15.96	training collapse
No norm -> BatchNorm (AlexNet-64)	CIFAR-10 / 10	73.52	79.93	+6.41	+28% epoch time
	Tiny / 10	16.46	30.15	+13.69	+7% epoch time
No aug -> aug (AlexNet-64 + BN)	CIFAR-10 / 10	79.93	78.73	-1.20	gap 4.93 -> 2.02
	CIFAR-10 / 15	81.18	82.83	+1.65	gap 11.72 -> 2.99
	Tiny / 10	30.15	29.20	-0.95	gap 0.3 -> -1.2
Augmentation during norm study (None / LRN / BN)	CIFAR-10 / 10	73.52 / 75.14 / 79.93	73.27 / 74.71 / 78.73	-0.25 / -0.43 / -1.20	≈ 59 s/epoch (CPU-bound)
10 -> 15 epochs (BN, no aug / aug)	CIFAR-10	79.93 / 78.73	81.18 / 82.83	+1.25 / +4.10	aug benefits more
AlexNet-64 (None) -> Zoo AlexNet	CIFAR-10 / 10	73.52	65.85	-7.67	8.6 * params, 0.43 * FLOPs
	Tiny / 10	16.46	11.02	-5.44	
LRN on Tiny: seed 42 -> seed 43	Tiny / 10	0.50	**16.50**	**+16.00**	trains normally (≈ None, 16.46)
LRN on Tiny: lr 10^{-3} -> 10^{-4}	Tiny / 10	0.50	**21.62**	**+21.12**	F1 19.06; best non-BN AlexNet

Multi-Seed Evaluation (Mean ± Std)

Table 11. Multi-seed results on CIFAR-10 (10 epochs, no augmentation).

Model	Seed 42	Seed 123	Seed 999	Mean ± Std (%)
AlexNet-64 None, Val Acc	73.52	73.52	74.65	**73.90 ± 0.65**
AlexNet-64 None, Macro F1	73.48	73.16	74.38	**73.67 ± 0.63**
AlexNet-64 LRN, Val Acc	75.14	76.24	75.58	**75.65 ± 0.55**
AlexNet-64 LRN, Macro F1	75.26	76.05	75.74	**75.68 ± 0.40**
AlexNet-64 BN, Val Acc	79.93	78.72	79.04	**79.23 ± 0.63**
AlexNet-64 BN, Macro F1	80.06	78.25	79.32	**79.21 ± 0.91**

Each normalization variant was trained with three seeds (42, 123, 999); the standard deviation is the unbiased sample estimate (ddof = 1) from utils/seed.py. Seed-to-seed variation is 0.4–0.9 points. The ranking BN > LRN > None held for every seed. BatchNorm's advantage is large relative to this noise (+5.3 points over None, +3.6 over LRN). LRN's gain over no normalization is small but consistent: +1.8 points on average, positive for all three seeds (+0.9 to +2.7), and about three times the standard deviation.

Loss and Accuracy Curves for Training and Validation Data

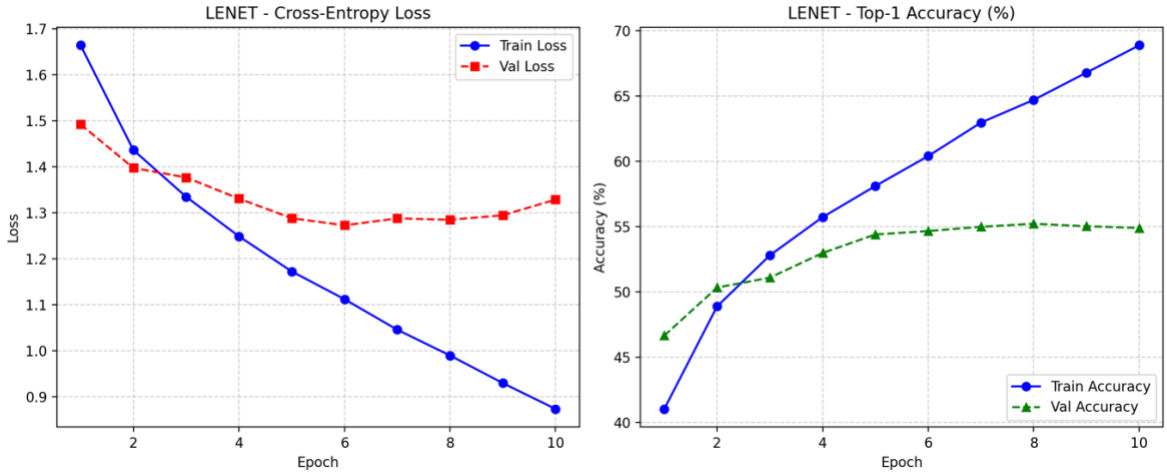


Figure 1. CIFAR-10: Side-by-side training and validation loss/accuracy - LeNet-5 - 10 epochs

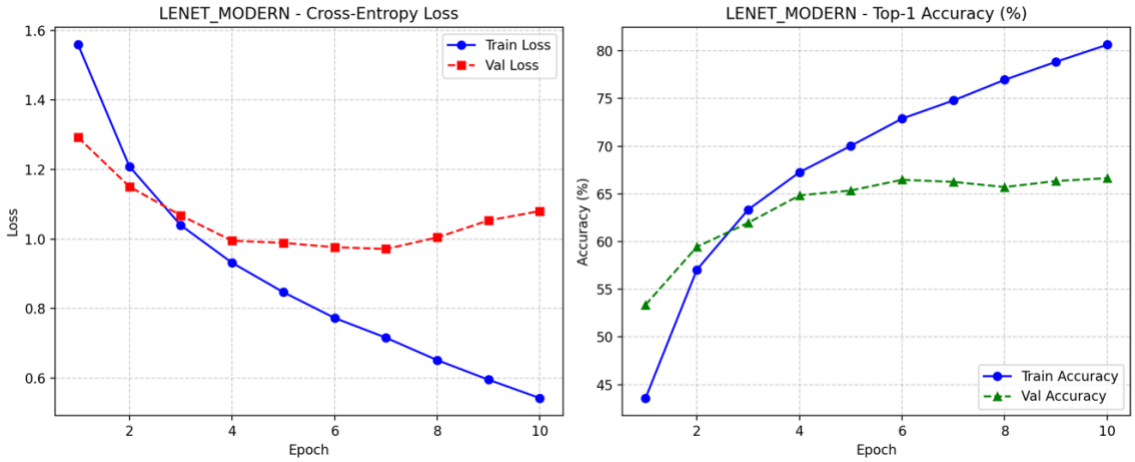


Figure 2. CIFAR-10: Side-by-side training and validation loss/accuracy - Modern LeNet - 10 epochs

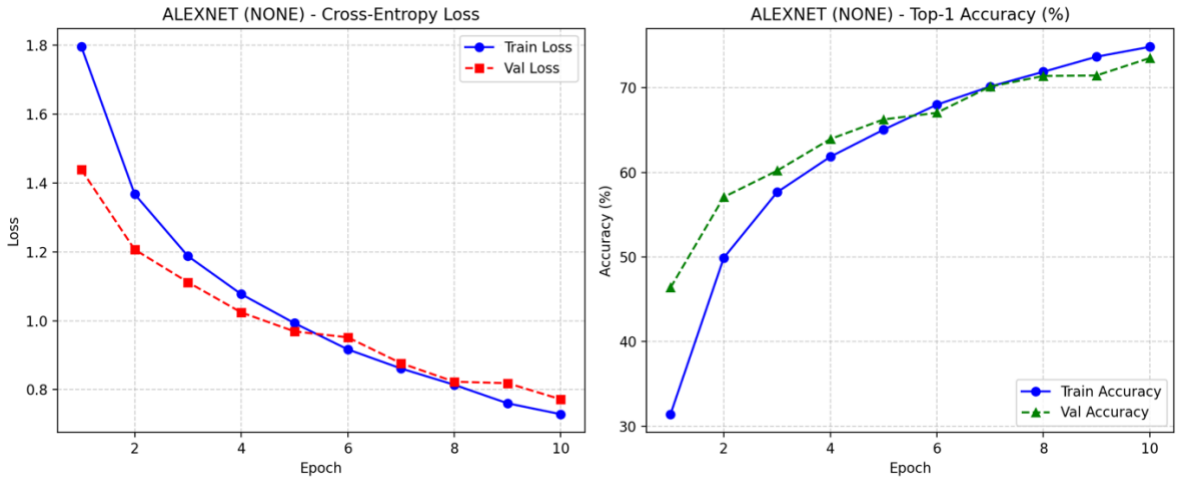


Figure 3. CIFAR-10: Side-by-side training and validation loss/accuracy - AlexNet-NONE- No Augmentation - 10 epochs

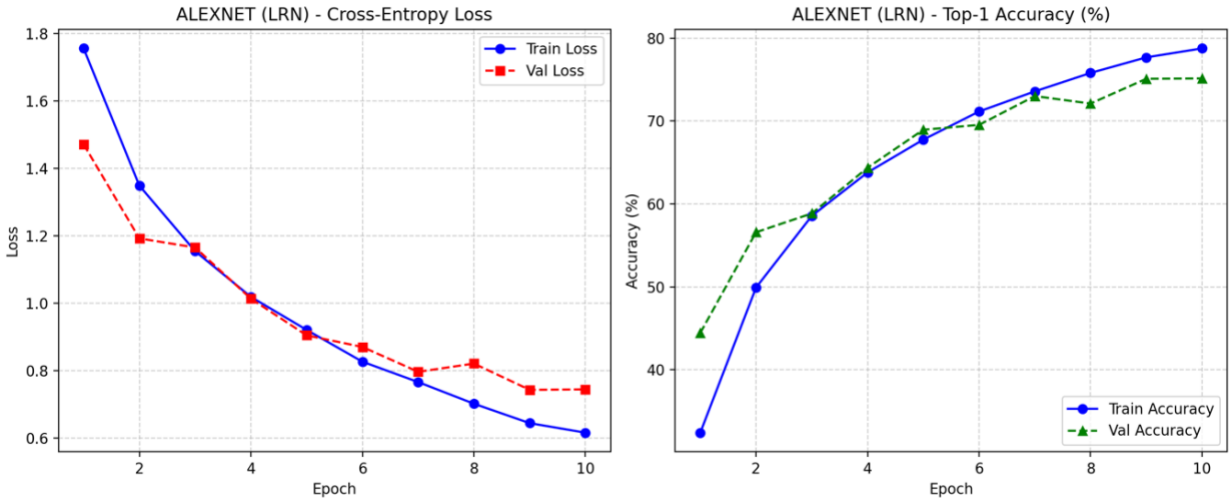


Figure 4. CIFAR-10: Side-by-side training and validation loss/accuracy - AlexNet-LRN- No Augmentation - 10 epochs

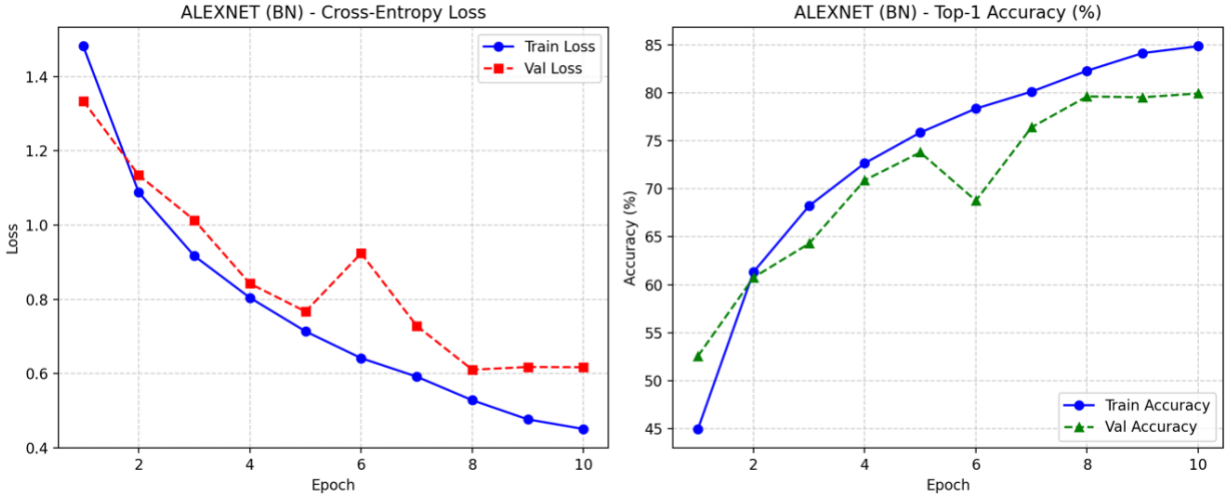


Figure 5. CIFAR-10: Side-by-side training and validation loss/accuracy - AlexNet-BN - No Augmentation - 10 epochs

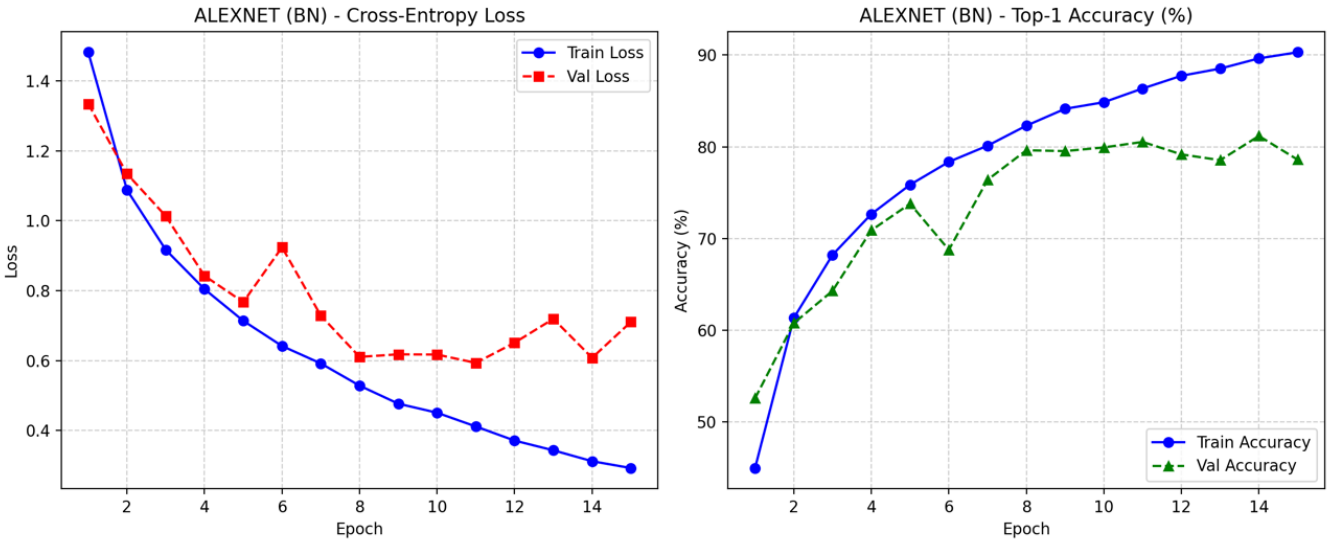


Figure 6. CIFAR-10: Side-by-side training and validation loss/accuracy - AlexNet-BN - No Augmentation – 15 epochs

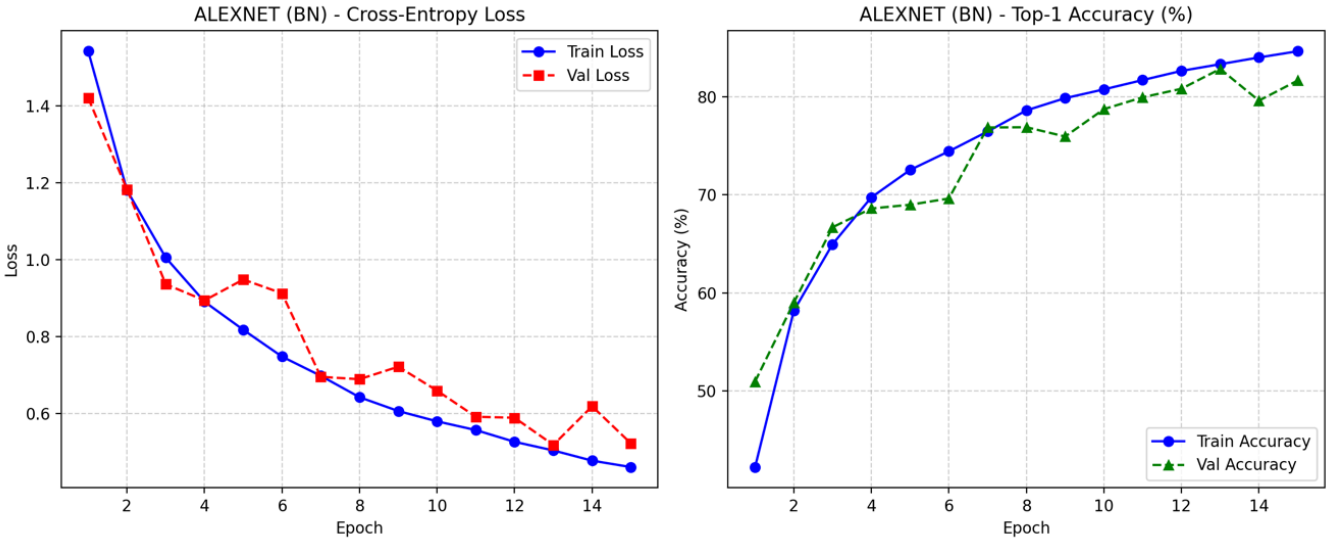


Figure 7. CIFAR-10: Side-by-side training and validation loss/accuracy - AlexNet-BN - With Augmentation 15 epochs

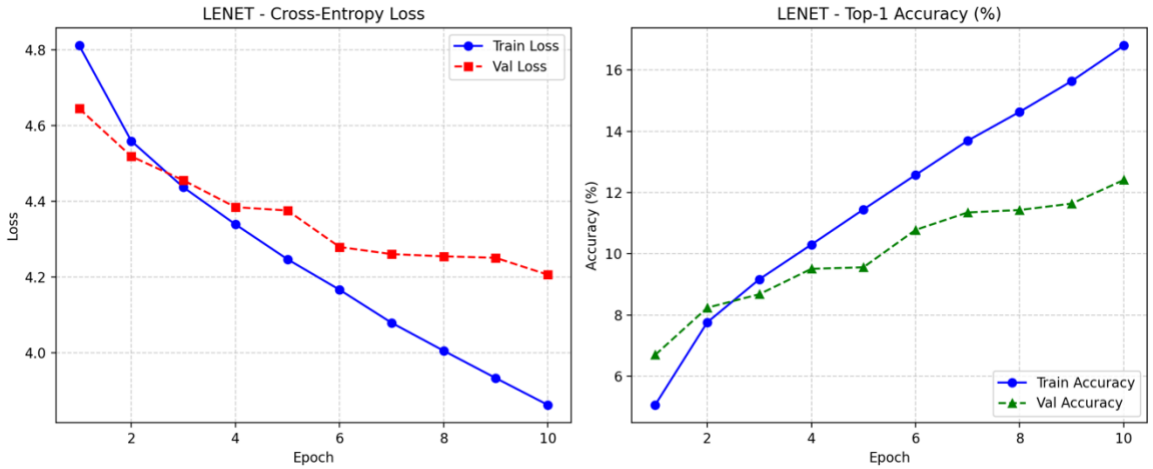


Figure 8. Tiny : Side-by-side training and validation loss/accuracy - LeNet-5 10 epochs

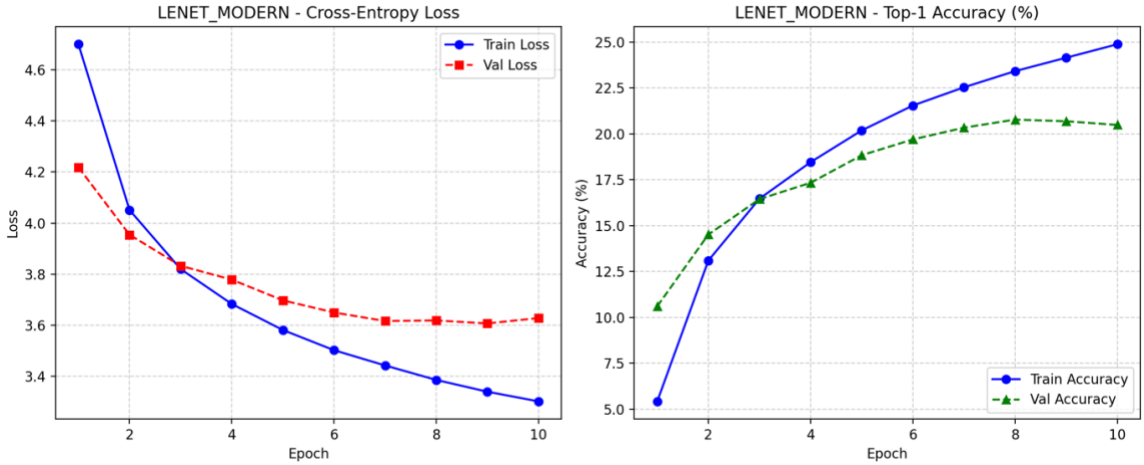


Figure 9. Tiny: Side-by-side training and validation loss/accuracy - Modern LeNet - 10 epochs

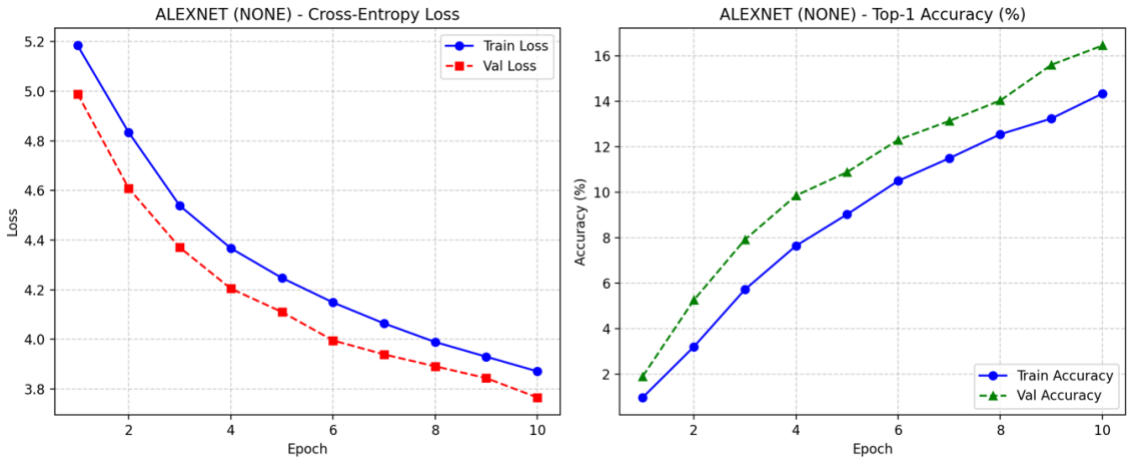


Figure 10. Tiny: Side-by-side training and validation loss/accuracy - AlexNet-NONE- No Augmentation - 10 epochs

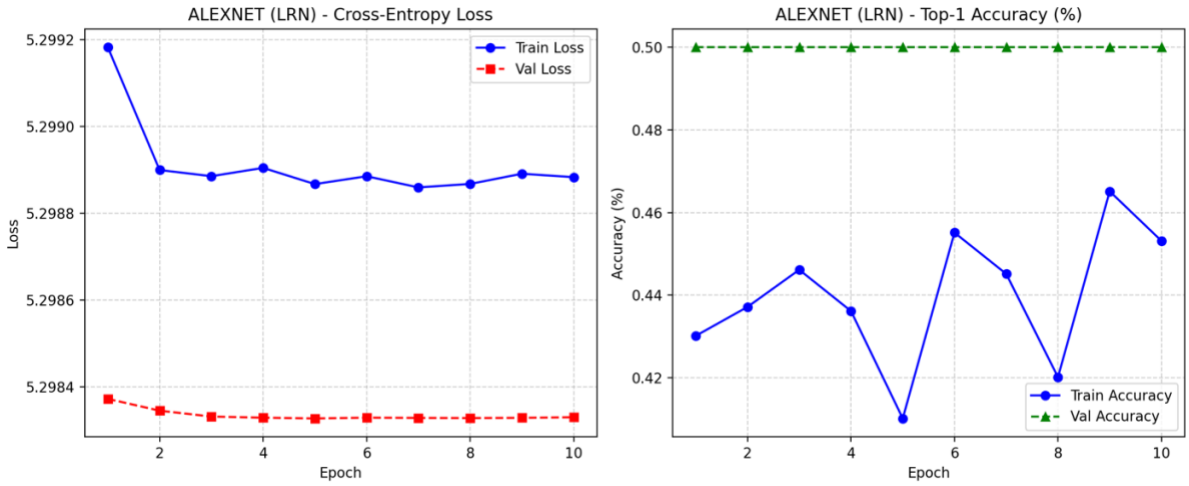


Figure 11. Tiny: Side-by-side training and validation loss/accuracy - AlexNet-LRN- No Augmentation 10 epochs

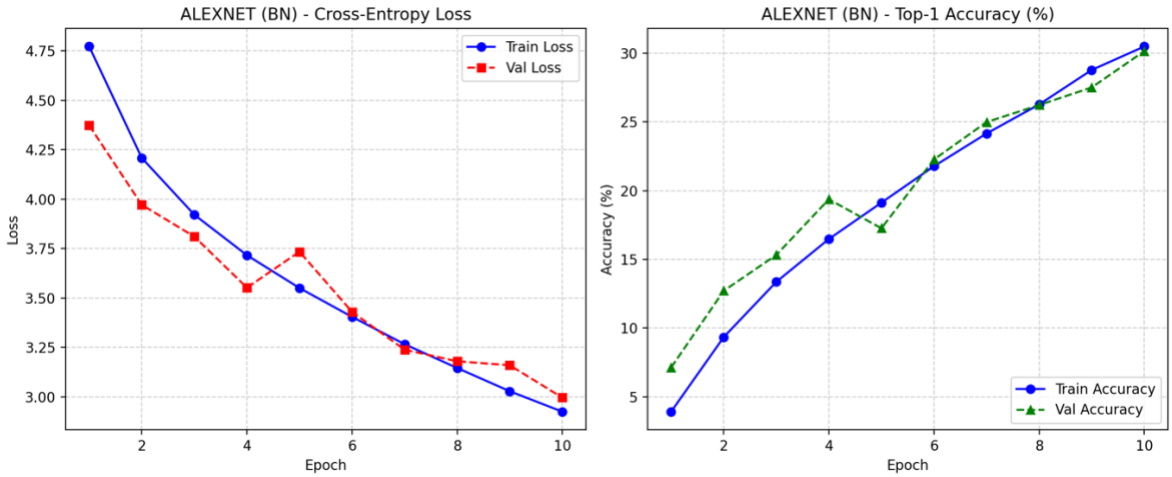


Figure 12. Tiny: Side-by-side training and validation loss/accuracy - AlexNet-BN - No Augmentation - 10 epochs

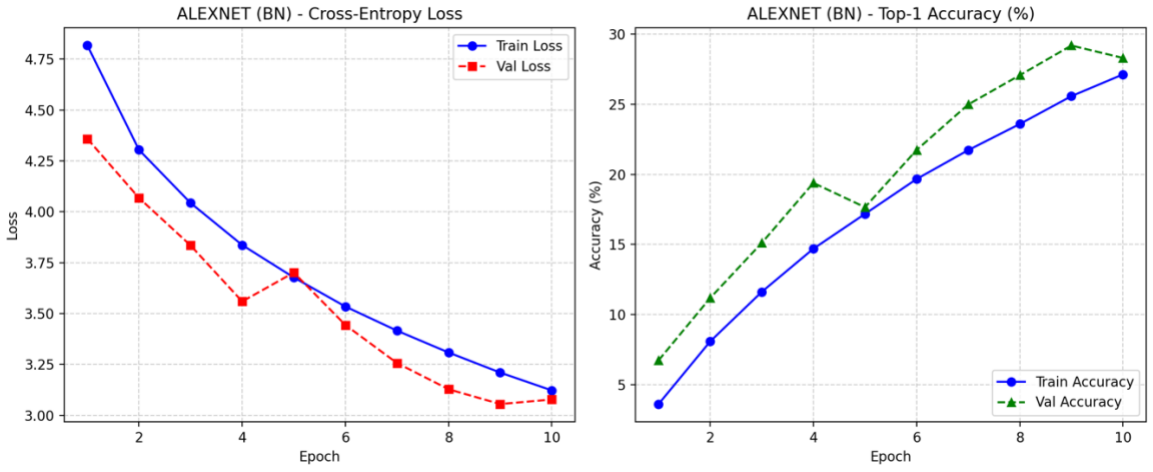


Figure 13. Tiny: Side-by-side training and validation loss/accuracy - AlexNet-BN - With Augmentation 10 epochs

Transforms output

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :    9.63 ms  (103,795 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   35.11 ms  ( 28,485 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  231.81 ms  (  4,314 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Figure 14. Output from demo_transforms.py showing data types, shapes, location, and memory

Dataset Class Balance and Feature Distribution

Both datasets are perfectly balanced: CIFAR-10 has 5,000 training images per class and Tiny ImageNet-200 has 500, giving an imbalance ratio of 1.00 and an entropy ratio of 1.0000 for both. Top-1 accuracy is unbiased, and macro F1 is reported alongside it. The CIFAR-10 channel means are [0.4923, 0.4831, 0.4474] and the standard deviations [0.2404, 0.2370, 0.2557]. Neighbouring channels are highly correlated (R–G 0.899, G–B 0.904).

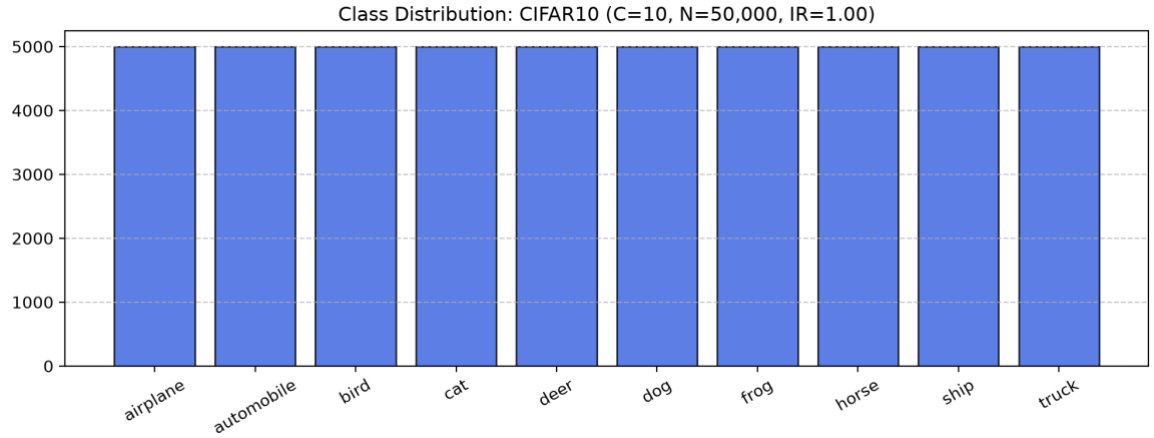


Figure 15. CIFAR-10 class distribution from utils/eda.py.

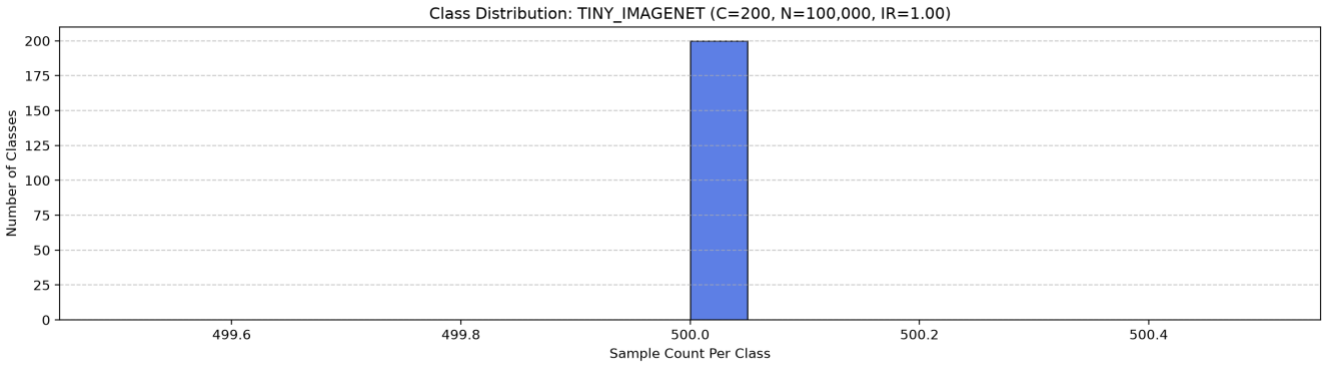


Figure 16. Tiny ImageNet class distribution from utils/eda.py.

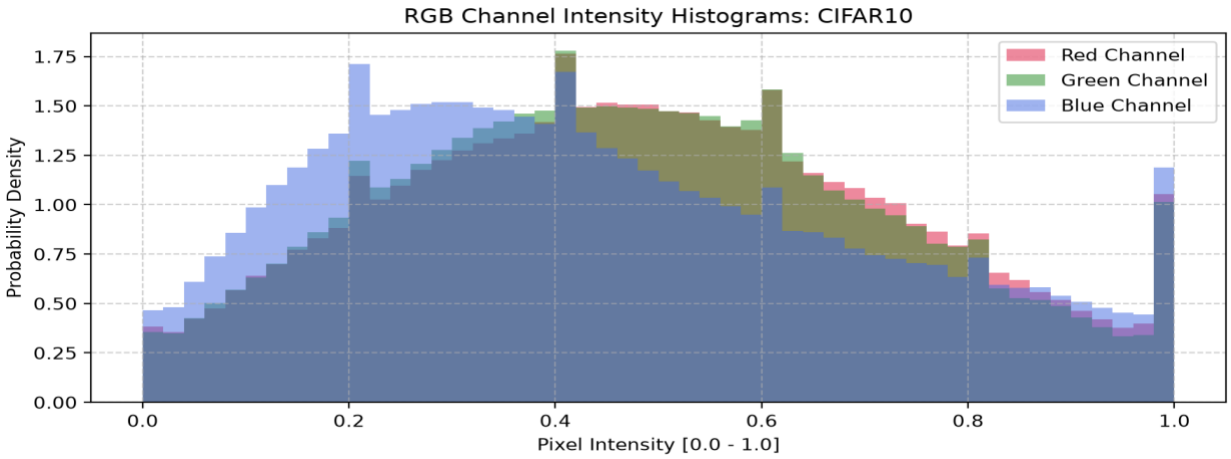


Figure 17. CIFAR-10 per-channel pixel-intensity histograms from utils/eda.py.

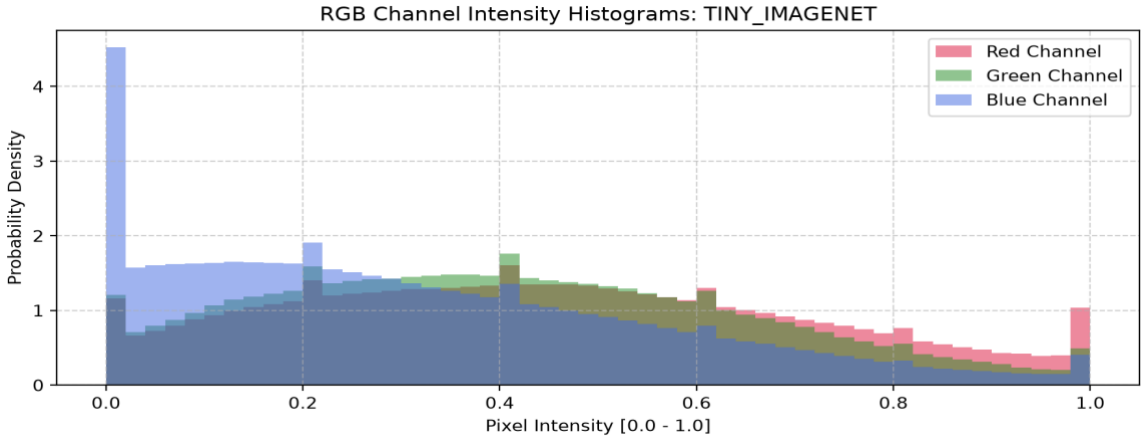


Figure 18. Tiny ImageNet per-channel pixel-intensity histograms from utils/eda.py.

Model Complexity Benchmark (compare_models.py)

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (Tesla T4)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| --------------------------- |----------- |--------- |---------- |------------- |------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.36         | 2794.8             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.39         | 2587.8             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 2.10         | 475.9              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 1.66         | 602.6              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 1.51         | 661.6              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 1.92         | 519.8              |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

Figure 19. Terminal output of python compare_models.py

Technical Analysis & Discussion

LeNet vs. Modern LeNet

The modernized LeNet improved accuracy by +11.4 points on CIFAR-10 (66.65% vs. 55.23%) and +8.4 points on Tiny ImageNet (20.77% vs. 12.41%). Two mechanisms explain the gain. First, **ReLU does not saturate**. The derivative of Tanh, $1 - \tanh^2(x)$, approaches zero for $|x| > 2$, and backpropagation multiplies such factors layer by layer, so gradients reaching the early layers shrink (the vanishing-gradient problem). ReLU has a derivative of exactly 1 for every active unit, which passes the gradient through unattenuated and speeds up convergence. The effect is visible from the first epoch: 53.4% vs. 46.7% validation accuracy. Second, **max pooling preserves the strongest local response** where average pooling blurs a strong edge activation with its weak neighbours. Overlapping windows with padding also keep a larger 15 * 15 map, which gives FC1 more spatial information.

The modernized model has 28% more FLOPs (9.19 vs. 7.17 M) yet trains at the same 12.9 s per epoch (≈ 3,850 images/s). For networks this small, the GPU waits for the data loader, so the extra arithmetic is effectively free. Both LeNets overfit after epochs 6–7: their validation loss reaches its minimum (1.2724 at epoch 6; 0.9715 at epoch 7) and then rises while training loss keeps falling.

Normalization Study: None vs. LRN vs. BatchNorm

Performance improvement: BatchNorm gave the highest validation accuracy and macro F1 on both datasets: 79.93% / 80.06% vs. 73.52% / 73.48% without normalization and 75.14% / 75.26% with LRN on CIFAR-10, and 30.15% vs. 16.46% on Tiny ImageNet. In a deep network the distribution of each layer's input shifts as the preceding layers update (internal covariate shift). For every channel, BatchNorm standardizes activations with the mini-batch statistics, and then rescales them with learnable parameters. This keeps every layer's activation on a stable scale, which smooths the optimization landscape and makes gradient steps more reliable. LRN only divides

each activation by the energy of its neighbouring channels. That creates competition between channels but does not control their overall scale or offset so its gain is marginal: +1.8 ± 0.6 points over three seeds, consistent but small next to BN's +5.3. Part of BN's advantage may also come from coverage, since BN normalizes all five convolutions while LRN normalizes only the first two.

Convergence stability: BN had the lowest training loss at every epoch and the fastest start (first-epoch validation accuracy 52.6% vs. ≈ 45%). Its training curve is the smoothest. Its validation curve is the bumpiest (a dip to 68.8% at epoch 6), consistent with evaluation using running-average statistics that lag the rapidly changing weights. On Tiny ImageNet the seed-42 LRN run failed to train: its loss stayed at 5.298 = ln(200), the uniform-prediction loss, for all ten epochs. To diagnose this, we re-ran it twice. With seed 43 it trained normally (16.50%, matching the unnormalized model's 16.46%), and with a 10× smaller learning rate (10^{-4}) it reached 21.62% / 19.06% macro F1, the best non-BN AlexNet on Tiny ImageNet. The failure was therefore initialization-dependent rather than an implementation error. It is consistent with a dying-ReLU collapse: at lr = 10^{-3}, without normalization and with LRN scaling activations down, an unlucky early update pushed the pre-activations negative, every ReLU output zero, and no gradient reached the convolutions. The unnormalized model came close to the same failure (1.9% after epoch 1) but recovered. BN, which re-centres and rescales every layer, trained stably at lr = 10^{-3} (30.15%), showing that it removes this sensitivity to the seed and learning rate. Because the unnormalized model was not re-run at 10^{-4}, the 21.62% result mixes the learning-rate effect with that of LRN.

Computational overhead: BN adds 1.27 MFLOPs and LRN adds none counted, yet epoch time grew from 15.5 s (None) to 19.9 s (BN, +28%) and 22.6 s (LRN, +46%). LRN is a memory-bound cross-channel operation, while BN is a highly optimized cuDNN kernel. When the CPU augmentation pipeline was the bottleneck (teammate runs with augmentation), all three variants took ≈ 59 s per epoch and the overhead was hidden. Given its large accuracy and stability benefit, BN's overhead is clearly worthwhile.

Data Augmentation and the Generalization Gap

Generalization gap reduction: yes. Over 15 epochs on CIFAR-10, the unaugmented model reached 90.31% training vs. 78.59% validation accuracy (Δ = 11.72), while the augmented model reached 84.64% vs. 81.65% (Δ = 2.99). At 10 epochs the gap fell from 4.93 (84.86 / 79.93) to 2.02 (80.75 / 78.73). **Validation improvement: yes, with enough training.** The best validation accuracy / F1 rose from 81.18% / 81.08% to 82.83% / 82.90% at 15 epochs. Without augmentation, validation loss rose after epoch 11 (0.5933 -> 0.7104) while training loss kept falling. With augmentation it kept decreasing (0.5181 at epoch 13).

Random crops, flips and colour jitter are label-preserving: a shifted, mirrored or slightly brighter cat is still a cat. Because the network never sees the same image twice, it cannot memorize pixel-exact positions. It is forced to learn features that are invariant to position, orientation and colour, and these transfer to unseen images. At 10 epochs, however, augmentation slightly lowered accuracy (-1.20 on CIFAR-10, -0.95 on Tiny ImageNet). The harder, randomized training distribution is fitted more slowly, and a regularizer only pays off once the unregularized model starts to overfit. Augmentation also increased epoch time by 54% on CIFAR-10 and 98% on Tiny ImageNet, because the transforms run on the CPU in the data loader.

Dual-Dataset Benchmark: CIFAR-10 vs. Tiny ImageNet-200

Above random chance: yes. Every model except the collapsed LRN variant learned discriminative features on Tiny ImageNet. AlexNet-64 + BN reached 30.15%, which is 60 * the 0.5% chance level, and even LeNet-5 reached

12.41%. **Cross-dataset comparison.** Accuracies are 2.7–6 * lower than on CIFAR-10, and macro F1 falls 1.5–3 points below accuracy, whereas on CIFAR-10 the two are nearly equal. This indicates that some of the 200 classes are rarely predicted. Tiny ImageNet is harder because it has 20 * more classes, 10 * fewer training images per class (500 vs. 5,000) and many fine-grained, visually similar categories. Normalization matters more on the harder task: BN's advantage over no normalization grew from +6.4 to +13.7 points, and the unnormalized AlexNet fell behind the much smaller modernized LeNet (16.46% vs. 20.77%).

Overfitting trajectory. Validation loss did *not* diverge earlier than on CIFAR-10. For every model except the modernized LeNet it was still decreasing at epoch 10, training accuracies were only 15–30%, and the AlexNet gaps were close to zero (BN: 0.3 points). The models are still under-fitting: with 200 classes, ten epochs are not enough to fit the training set. Only the modernized LeNet, the smallest fast-converging model, plateaued (from epoch 7) as it ran out of capacity. Overfitting would be expected to appear with substantially longer training.

Custom AlexNet-64 vs. PyTorch Zoo AlexNet

The 57 M-parameter Zoo AlexNet was 7.7 points less accurate than AlexNet-64 without normalization on CIFAR-10 (65.85% vs. 73.52%) and 5.4 points less accurate on Tiny ImageNet. Its feature map collapses to 1 * 1 on 64 * 64 inputs, so spatial layout is lost before the classifier, and 37.7 M FC1 weights act on 36 replicated copies of 256 values. Parameter count alone does not buy accuracy; the architecture must match the input resolution. The comparison also shows that parameters, FLOPs and time are distinct quantities: the Zoo model has 8.6 * the parameters and 0.43 * the FLOPs of AlexNet-64, with a similar epoch time and latency.

Computational Complexity: FLOPs, Time and Latency

FLOPs are dominated by convolutions because each filter is applied at every position: in LeNet-5 the two convolutions account for 3.2 of 3.6 M MACs while holding under 1% of the parameters. Training time follows whichever resource is the bottleneck. For small models and augmented runs this is the CPU data pipeline, so FLOPs barely matter. For AlexNet-64 without augmentation it is the GPU, where memory-bound layers such as LRN cost more than their FLOP count suggests. Inference latency at batch size 1 is dominated by kernel-launch overhead. AlexNet-64 has 60 * the FLOPs of LeNet-5 but is only ≈ 5.7 * slower (≈ 2.0 vs. 0.35 ms). Latency differences of ≈ 0.5 ms between the AlexNet variants were not reproducible across two runs (LRN: 2.15 then 1.66 ms), so they are within measurement noise. Small FLOP differences therefore matter far less on a parallel GPU than on a sequential CPU or an edge microcontroller, where every operation and every byte of memory counts.

Limitations

Apart from the three-seed normalization study, results come from a single seed (42) and a fixed 10-epoch budget, so differences of 1–2 points elsewhere are not statistically established. Normalization type is confounded with coverage (five BN layers vs. two LRN layers), and the in-lab LeNet modernization changes activation, pooling and padding together. Wall-clock times depend on the Colab runtime assigned (the same BN configuration took ≈ 20 s per epoch on our runtime and ≈ 27 s on our teammate's), so only times from the same runtime are compared directly. The learning rate and batch size were not tuned per architecture.